Lecture 10: Time Complexity

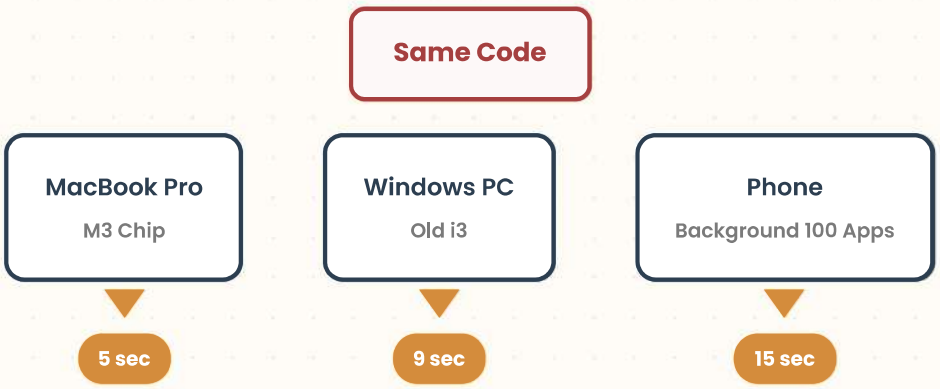
Is lecture mein hum seekhenge ki algorithm ki efficiency ko kaise measure karte hain. Time Complexity woh tool hai jo batata hai ki tumhara code input ke size ke saath kaise behave karega. Yeh concept har DSA interview aur competitive programming round mein pucha jata hai.

1. Why Do We Need Time Complexity?

Ek problem ko solve karne ke **multiple approaches** hote hain. Lekin kaunsa approach best hai? Iska jawab sirf "code chal raha hai" se nahi milta. Hamein ek standard metric chahiye jo machine, language, aur environment se independent ho.

A. The Machine Problem

Same code alag-alag machines pe alag speed mein run karta hai:



> **INTUITION**
>
> Phone mein background mein 100 apps chal rahe hain toh woh slow hoga. Same code powerful machine pe fast chalega. Toh agar hum sirf "seconds" mein measure karein, toh result har baar alag aayega. Hamein ek **machine-independent** metric chahiye.

B. The Language Problem

Same algorithm Python mein likhne pe C++ se zyada time lagta hai. Python interpreted hai, C++ compiled hai. Toh language bhi execution time affect karti hai.

IMPORTANT

Time Complexity machine speed , programming language , aur background processes se **independent** hoti hai. Yeh sirf algorithm ke logic pe depend karti hai — kitne operations perform ho rahe hain input size ke hisaab se.

2. What is Time Complexity?

FORMAL DEFINITION

Time Complexity is a function that describes how the number of operations an algorithm performs grow as the input size increases .

Matlab, agar input size n badhta hai, toh algorithm ke operations kitne guna badhenge? Yeh growth pattern Time Complexity define karta hai.

3. Operation Counting Method

Sabse pehla aur most intuitive method: count the operations . Har line ka execution count karo aur total operations nikaalo.

Example: Sum of n Numbers (Array Traversal)

```
int arr[] = {10, 20, 30, 40, 50};   // n = 4 elements
int sum = 0;

for(int i = 0; i < 4; i++) {
    sum += arr[i];
}

cout << sum;
```

DRY RUN / OPERATION COUNT

Har iteration mein kitne operations hote hain:

Iteration (i)	Condition Check	Sum Update	Increment	Total Ops
i = 0	1 (i < 4)	1 (sum += arr[0])	1 (i++)	3
i = 1	1	1	1	3
i = 2	1	1	1	3
i = 3	1	1	1	3
i = 4	1 (i < 4 → false)	0	0	1
			Grand Total:	**14 ops**

```
Total Operations = 3n + 2
```

Jahan `n = 4` ke liye: `3(4) + 2 = 14` operations.

VERIFICATION WITH DIFFERENT N

n	Formula: 3n + 2	Actual Operations
4	3(4) + 2 = 14	14 ✓
5	3(5) + 2 = 17	17 ✓
6	3(6) + 2 = 20	20 ✓

4. Big-O Notation: Ignoring Constants & Lower Terms

Operation counting se exact formula milta hai, lekin DSA mein hamein exact count nahi chahiye. Hamein growth pattern chahiye. Isliye hum **Big-O Notation** use karte hain.

A. Why Ignore Constants?

WORKFLOW / ALGORITHM

Step-by-Step Simplification:

1. Algorithm A ki T.C = $3n + 2$
2. Algorithm B ki T.C = $2n^2 + 6$
3. Dono mein constants (3, 2, 6) hain jo machine speed pe depend karte hain
4. Constants ko ignore karo → $3n + 2$ becomes n
5. Lower order terms ko ignore karo → $n^2 + n$ becomes n^2

$$3n + 2$$

↓ Ignore Constants

$$n$$

↓ Big-O

$$O(n)$$

> **Key Insight:** *Jab n bahut bada hota hai (1 million, 1 billion), toh constants aur lower order terms ka contribution negligible ho jata hai. $3n + 2$ aur $100n + 500$ dono same $O(n)$ category mein aate hain kyunki dono linear growth follow karte hain.*

B. Why Ignore Lower Order Terms?

$$5N^3 + 2N^2 + 3N + 26$$

↓ Dominant Term Rule

$$N^3 + N^2 + N$$

↓ Ignore Lower Terms

$$O(N^3)$$

IMPORTANT

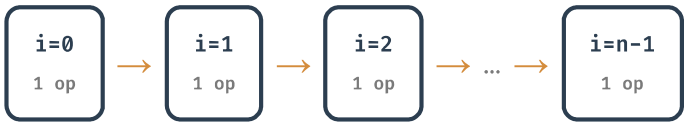
Big-O mein hamesha **worst-case scenario** consider karte hain. Hamesha woh term lo jo input size ke sabse zyada power pe hai — kyunki jab $n \to \infty$ hota hai, toh wohi term sabse zyada dominate karta hai.

5. Pattern 1: Single Loop — O(n)

Sabse common pattern. Loop `0` se `n-1` tak chalta hai, har baar ek operation perform hota hai.

```cpp
for(int i = 0; i < n; i++) {
    // Some O(1) work
    cout << "hello";
}
```

DRY RUN / VISUALIZATION



Total = n times × 1 op = `O(n)`

6. Pattern 2: Loop with Step Size — Still O(n)

Loop increment 1 se zyada ho sakta hai, lekin agar step size constant hai, toh bhi complexity `O(n)` rehti hai.

A. Step Size = 2

```cpp
for(int i = 0; i < n; i = i + 2) {
    cout << "hello";
}
```

DRY RUN / VISUALIZATION

Loop `n/2` baar chalega. Formula: `n/2` operations.

Big-O mein: `n/2` → constant `1/2` ignore → `O(n)`

B. Step Size = 3

```cpp
for(int i = 0; i < n; i = i + 3) {
    cout << "hello";
}
```

DRY RUN / VISUALIZATION

Loop `n/3` baar chalega. Formula: `n/3` operations.

Big-O mein: `n/3` → constant `1/3` ignore → `O(n)`

Key Insight: *Chahe step size 2 ho, 3 ho, ya 100 ho — agar step size* ***constant*** *hai (input `n` pe depend nahi karta), toh complexity hamesha* `O(n)` *hi hogi. Constant factor Big-O mein matter nahi karta.*

7. Pattern 3: Multiple Sequential Loops — O(n)

Multiple loops ek ke neeche ek (sequential) hain, toh complexities **add** hoti hain.

```cpp
// Loop 1: runs n times
for(int i = 0; i < n; i++) {
    cout << "hello";
}

// Loop 2: runs n/5 times
for(int j = 0; j < n; j = j + 5) {
    cout << "NO";
}
```

DRY RUN / VISUALIZATION

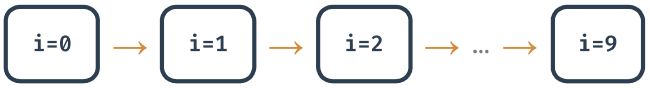
`6n/5` → constant `6/5` ignore → `O(n)`

8. Pattern 4: Constant Loop — O(1)

Loop ka count input `n` se bilkul independent ho, toh complexity **constant** hoti hai.

```
for(int i = 0; i < 10; i++) {
    cout << "hello";
}
```

DRY RUN / VISUALIZATION



Loop exactly **10 times** chalega — chahe `n = 1` ho ya `n = 1 million`.

Input size se independent → `O(1)`

INTUITION

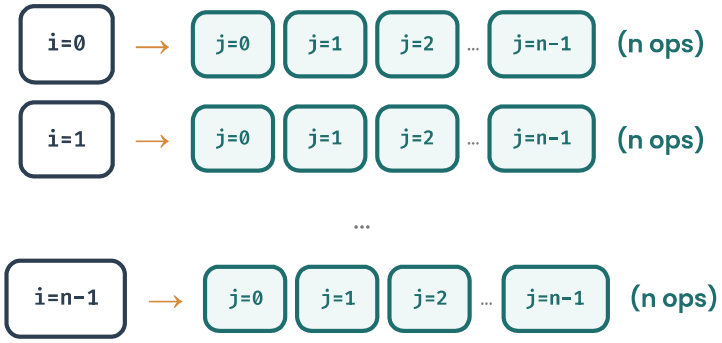
`O(1)` matlab constant time. Tumhara code chahe input 1 ka ho ya 1 crore ka, execution time same rahega. Iska example array ka direct index access `arr[5]` bhi hai — yeh bhi `O(1)` hai.

9. Pattern 5: Nested Loops (Independent) — O(n²)

Jab ek loop ke andar doosra loop hota hai, toh complexities **multiply** hoti hain.

```
for(int i = 0; i < n; i++) {          // Outer: n times
    for(int j = 0; j < n; j++) {      // Inner: n times
        cout << "hello";
    }
}
```

DRY RUN / VISUALIZATION



Total = n × n = n² → O(n²)

Fixed Bound Nested Loops — O(1)

Agar dono loops fixed iterations ke hain, toh result constant hota hai:

```
for(int i = 0; i < 3; i++) {          // 3 times
    for(int j = 0; j < 3; j++) {      // 3 times
        cout << "hello";
    }
}
// Total = 3 × 3 = 9 (constant) → O(1)
```

DRY RUN / VISUALIZATION

3 × 3 = 9 → fixed number, input independent → O(1)

10. Pattern 6: Nested Loops (Dependent Inner Bound) — O(n²)

Inner loop ka bound outer loop variable pe depend karta hai:

```
for(int i = 0; i < n; i++) {          // Outer: n times
    for(int j = 0; j <= i; j++) {     // Inner: 0 to i
        cout << "hello";
    }
}
```

DRY RUN / VISUALIZATION

Outer i	Inner j runs	Operations
0	0	1
1	0, 1	2
2	0, 1, 2	3
...	...	...
n-1	0 to n-1	n
	Total:	**1 + 2 + 3 + ... + n = n(n+1)/2**

$n(n+1)/2 = n^2/2 + n/2 \rightarrow$ ignore constants $\rightarrow$ $O(n^2)$

Key Insight: *Triangle pattern wale nested loops ($j \leq i$) bhi $O(n^2)$ hote hain. Exact operation count $n(n+1)/2$ hai, lekin Big-O mein sirf dominant term n^2 matter karta hai. Yeh pattern sorting algorithms (Bubble Sort, Selection Sort) mein common hai.*

11. Pattern 7: Direct Formula vs Loop — O(1) vs O(n)

Same problem ko solve karne ke do tarike — ek loop se, ek formula se:

Approach 1: Loop (O(n))

```
int sum = 0;
for(int i = 1; i <= n; i++) {
    sum += i;
}
cout << sum;
// T.C = O(n)
```

Approach 2: Formula (O(1))

```
int sum = n * (n + 1) / 2;
cout << sum;
// T.C = O(1)
```

BEST PRACTICE / OPTIMIZATION

Mathematical formulas se direct computation hamesha `O(1)` hota hai. Jab bhi tumhe sum, average, ya koi mathematical series ka result chahiye, toh pehle formula check karo. Loop se nikaalna `O(n)` hoga, lekin formula se `O(1)` — yeh optimization interviews mein impress karta hai.

12. Complexity Cheat Sheet

Pattern	Code Structure	Complexity	Common Use Case
Single Loop	`for(i=0; i`	`O(n)`	Array traversal, linear search
Loop with Step	`for(i=0; i`	`O(n)`	Stepping through array
Sequential Loops	Loop A + Loop B	`O(n)`	Multiple independent passes
Constant Loop	`for(i=0; i<10; i++)`	`O(1)`	Fixed iterations
Nested Loops	for(i) { for(j) }	`O(n²)`	Bubble sort, matrix traversal
Triangle Nested	for(i) { for(j≤i) }	`O(n²)`	Selection sort, pair generation
Direct Formula	`sum = n*(n+1)/2`	`O(1)`	Mathematical computation

13. Key Takeaways

TAKEAWAY

- **Machine Independent:** Time Complexity machine speed, language, aur environment se independent hoti hai. Yeh algorithm ki efficiency batati hai.
- **Operation Counting:** Har line ke operations count karo, formula banao, phir Big-O notation mein convert karo.
- **Ignore Constants:** $3n + 2 \rightarrow O(n)$. Constants execution speed pe depend karte hain, algorithm ki nature pe nahi.
- **Dominant Term:** $5N^3 + 2N^2 + 3N + 26 \rightarrow O(N^3)$. Hamesha sabse bada power wala term lo.
- **Single Loop:** Har single loop (chahe step 1 ho ya 100) `O(n)` hota hai.
- **Nested Loops Multiply:** Outer n × Inner n = `O(n²)`. Sequential loops add hote hain.
- **Fixed Loops:** Agar loop count input se independent hai, toh `O(1)`.
- **Formula Optimization:** Mathematical formula se direct computation `O(1)` hota hai — hamesha prefer karo.

INTERVIEW TIP

Interview mein frequently pucha jata hai: *"What is the time complexity of this code?"* Code dekhke immediately pattern identify karo:

- Single loop → `O(n)`
- Nested loops with same bound → `O(n²)`
- Nested loops with $j \leq i$ → `O(n²)` (triangle pattern)
- No loop, direct access → `O(1)`

Practice karte waqt **operation counting** method se start karo, phir gradually direct pattern recognition develop karo.

PATTERN TRANSFER

Yeh Time Complexity concepts aage har DSA topic mein use honge:

- **Arrays:** Linear search `O(n)`, Binary search `O(log n)`
- **Sorting:** Bubble/Selection `O(n²)`, Merge/Quick `O(n log n)`
- **Two Pointers:** Single pass `O(n)` instead of nested `O(n²)`
- **Sliding Window:** `O(n)` by avoiding nested loops
- **DP:** Often `O(n²)` or `O(n³)` based on state transitions

Time Complexity analysis woh lens hai jo tumhe optimal solution ki taraf le jaata hai.